Week 1

Problem 1: Finding Complexity using Counter Method

```
#include<stdio.h>
int counter=0;
int main()
{
    int i=1,n;
    counter++;
    int s=1;
    counter++;
    scanf("%d",&n);
    while(s<=n)
    {
        counter++;
        i++;
        counter++;
        s+=i;
        counter++;
    }
    counter++;
    printf("%d",counter);
}
```

Problem 2: Finding Complexity using Counter Method

```
#include<stdio.h>
int count=0;
int main(){
    int n;
    scanf("%d",&n);
    count++;
    if(n==1){
        //printf("*");
    }
    else{
        for(int i=1;i<=n;i++){
            count++;
            for(int j=1;j<=n;j++){
                count++;
                count++;
                //printf("*");
                count++;
                //printf("*");
                break;
            }
            count++;
        }
        count++;
    }
    printf("%d",count);
}
```

Problem 3: Finding Complexity using Counter Method

```
#include<stdio.h>
int count=0;
int factor(int num){
    for(int i=1;i<=num;++i){
        count++;
        count++;
        if(num%i==0){
            count++;
            //printf("%d ",i);
        }
    }
    count++;
    return 0;
}
int main(){
    int num;
    scanf("%d",&num);
    factor(num);
    printf("%d",count);
```

```
}
```

Problem 4: Finding Complexity using Counter Method

```
#include<stdio.h>
void function(int n){
    int count=0;
    count++;
    for(int i=n/2;i<n;i++){
        count++;
        for(int j=1;j<n;j=2*j){
            count++;
            for(int k=1;k<n;k=k*2){
                count++;
                count++;
            }
            count++;
        }
        count++;
    }
    count++;
    printf("%d",count);
}
int main(){
    int n;
    scanf("%d",&n);
    function(n);
}
```

Problem 5: Finding Complexity using Counter method

```
#include<stdio.h>
int count=0;
void reverse(int n){
    int rev=0,remainder;
    count++;
    while(n!=0){
        count++;
        remainder=n%10;
        count++;
        rev=rev*10+remainder;
        count++;
        n/=10;
        count++;
    }
    count++;
    count++;
    //printf(rev)
    printf("%d",count);
}
int main(){
    int n;
    scanf("%d",&n);
    reverse(n);
}
```

Week 2

1-G-Coin Problem

```
#include<stdio.h>
int main(){
    int n;
    int coins[]={1000,500,100,50,20,10,5,2,1};
    int count=0;
    scanf("%d",&n);
    for(int i=0;i<9;i++){
        while(n>=coins[i]){
            n=n-coins[i];
            count++;
        }
    }
    printf("%d",count);
    return 0;
}
```

2-G-Cookies Problem

```
#include<stdio.h>
int main(){
    int n,m,i,j,ans=0;
    scanf("%d",&n);

    int g[n];
    for(i=0;i<n;i++)
        scanf("%d",&g[i]);

    scanf("%d",&m);

    int s[m];
    for(i=0;i<m;i++)
        scanf("%d",&s[i]);

    for(i=0;i<n;i++)
        for(j=0;j<m;j++)
            if(s[j]>=g[i]){
                ans++;
                s[j]=0;
                break;
            }

    printf("%d",ans);
    return 0;
}
```

3-G-Burger Problem

```
#include <stdio.h>
#include <stdlib.h>

int compare(const void *a, const void *b) {
    return (*(int *)b - *(int *)a);  // Descending order
}

int main() {
    int n;
    scanf("%d", &n);

    int calories[n];

    for (int i = 0; i < n; i++) {
        scanf("%d", &calories[i]);
    }

    // Greedy: eat burgers in decreasing order of calories
    qsort(calories, n, sizeof(int), compare);

    long long total = 0;
    long long power = 1;  // 3^0

    for (int i = 0; i < n; i++) {
```

```
        total += calories[i] * power;
        power *= n;
    }

    printf("%lld\n", total);

    return 0;
}
```

4-G-Array Sum max

```
#include <stdio.h>
#include <stdlib.h>

int cmp(const void *a, const void *b) {
    return (*(int *)a - *(int *)b);
}

int main() {
    int n, i;
    long long sum = 0;

    scanf("%d", &n);

    int a[n];

    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);

    qsort(a, n, sizeof(int), cmp);

    for (i = 0; i < n; i++)
        sum += (long long)a[i] * i;

    printf("%lld", sum);

    return 0;
}
```

5-G-Product of Array elements-Minimum

```
#include <stdio.h>
#include <stdlib.h>

int asc(const void *a, const void *b) {
    return *(int *)a - *(int *)b;
}

int desc(const void *a, const void *b) {
    return *(int *)b - *(int *)a;
}

int main() {
    int n, i;
    long long sum = 0;

    scanf("%d", &n);

    int a[n], b[n];

    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);

    for (i = 0; i < n; i++)
        scanf("%d", &b[i]);

    qsort(a, n, sizeof(int), asc);
    qsort(b, n, sizeof(int), desc);

    for (i = 0; i < n; i++)
        sum += (long long)a[i] * b[i];

    printf("%lld", sum);

    return 0;
}
```

Week 3

1-Number of Zeros in a Given Array

```
#include <stdio.h>

int zero(int a[], int l, int h) {
    if (l > h) return 0;
    if (a[l] == 0) return h - l + 1;

    int m = (l + h) / 2;

    if (a[m] == 1)
        return zero(a, m + 1, h);
    return zero(a, l, m);
}

int main() {
    int n, a[100];
    scanf("%d", &n);

    for(int i=0; i<n; i++)
        scanf("%d", &a[i]);

    printf("%d", zero(a, 0, n-1));
    return 0;
}
```

2-Majority Element

```
#include <stdio.h>

int main() {
    int n, a[50000], count = 0, candidate;

    scanf("%d", &n);

    for(int i = 0; i < n; i++)
        scanf("%d", &a[i]);

    for(int i = 0; i < n; i++) {
        if(count == 0)
            candidate = a[i];

        if(a[i] == candidate)
            count++;
        else
            count--;
    }

    printf("%d", candidate);

    return 0;
}
```

3-Finding Floor Value

```
#include <stdio.h>

int floorValue(int a[], int l, int h, int x)
{
    if (l > h)
        return -1;

    int mid = (l + h) / 2;

    if (a[mid] == x)
        return a[mid];

    if (a[mid] < x)
        return (mid == h) ? a[mid] :
               (a[mid + 1] > x ? a[mid] : floorValue(a, mid + 1, h, x));

    return floorValue(a, l, mid - 1, x);
}
```

```
int main()
{
    int n, a[100], x;

    scanf("%d", &n);

    for(int i = 0; i < n; i++)
        scanf("%d", &a[i]);

    scanf("%d", &x);

    printf("%d", floorValue(a, 0, n - 1, x));

    return 0;
}
```

4-Two Elements sum to x

```
#include <stdio.h>

int find(int a[], int l, int r, int x) {
    if(l >= r) return 0;

    if(a[l] + a[r] == x) {
        printf("%d\n%d", a[l], a[r]);
        return 1;
    }

    if(a[l] + a[r] < x)
        return find(a, l + 1, r, x);

    return find(a, l, r - 1, x);
}

int main() {
    int n, a[100], x;

    scanf("%d", &n);
    for(int i=0; i<n; i++) scanf("%d", &a[i]);
    scanf("%d", &x);

    if(!find(a, 0, n-1, x))
        printf("No");

    return 0;
}
```

5-Implementation of Quick Sort

```
#include <stdio.h>

void q(int a[],int l,int h){
    if(l>=h) return;
    int i=l,j=h,p=a[l],t;
    while(i<j){
        while(a[i]<=p && i<h)i++;
        while(a[j]>p)j--;
        if(i<j){t=a[i];a[i]=a[j];a[j]=t;}
    }
    a[l]=a[j]; a[j]=p;
    q(a,l,j-1); q(a,j+1,h);
}

int main(){
    int n,a[100];
    scanf("%d",&n);
    for(int i=0;i<n;i++)scanf("%d",&a[i]);
    q(a,0,n-1);
    for(int i=0;i<n;i++)printf("%d ",a[i]);
}
```

Week 4

1-DP-Playing with Numbers

```
#include <stdio.h>

int main() {
    int n, i;
    long long dp[1000];

    scanf("%d", &n);

    dp[0] = 1;

    for (i = 1; i <= n; i++) {
        dp[i] = dp[i - 1];

        if (i >= 3)
            dp[i] += dp[i - 3];
    }

    printf("%lld", dp[n]);

    return 0;
}
```

2-DP-Playing with Chessboard

```
#include <stdio.h>

int main() {
    int n, i, j;
    int a[100][100], dp[100][100];

    scanf("%d", &n);

    for (i = 0; i < n; i++)
        for (j = 0; j < n; j++)
            scanf("%d", &a[i][j]);

    dp[0][0] = a[0][0];

    for (i = 1; i < n; i++)
        dp[i][0] = dp[i - 1][0] + a[i][0];

    for (j = 1; j < n; j++)
        dp[0][j] = dp[0][j - 1] + a[0][j];

    for (i = 1; i < n; i++) {
        for (j = 1; j < n; j++) {
            if (dp[i - 1][j] > dp[i][j - 1])
                dp[i][j] = dp[i - 1][j] + a[i][j];
            else
                dp[i][j] = dp[i][j - 1] + a[i][j];
        }
    }

    printf("%d", dp[n - 1][n - 1]);

    return 0;
}
```

3-DP-Longest Common Subsequence

```
#include <stdio.h>
#include <string.h>

int main() {
    char s1[100], s2[100];
    int dp[100][100];
    int i, j, m, n;

    scanf("%s", s1);
    scanf("%s", s2);
    m = strlen(s1);
```

```
    n = strlen(s2);

    for (i = 0; i <= m; i++)
        dp[i][0] = 0;

    for (j = 0; j <= n; j++)
        dp[0][j] = 0;

    for (i = 1; i <= m; i++) {
        for (j = 1; j <= n; j++) {
            if (s1[i - 1] == s2[j - 1])
                dp[i][j] = dp[i - 1][j - 1] + 1;
            else if (dp[i - 1][j] > dp[i][j - 1])
                dp[i][j] = dp[i - 1][j];
            else
                dp[i][j] = dp[i][j - 1];
        }
    }

    printf("%d", dp[m][n]);

    return 0;
}
```

4-DP-Longest non-decreasing Subsequence

```
#include <stdio.h>

int main() {
    int n, a[100], dp[100];
    int i, j, max = 1;

    scanf("%d", &n);

    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);

    for (i = 0; i < n; i++)
        dp[i] = 1;

    for (i = 1; i < n; i++) {
        for (j = 0; j < i; j++) {
            if (a[j] <= a[i] && dp[j] + 1 > dp[i])
                dp[i] = dp[j] + 1;
        }

        if (dp[i] > max)
            max = dp[i];
    }

    printf("%d", max);

    return 0;
}
```

Week 5

1-Finding Duplicates-O(n^2) Time Complexity,O(1) Space Complexity

```
#include<stdio.h>
int main(){
    int n;
    scanf("%d",&n);
    int a[n];
    for(int i=0;i<n;i++){
        scanf("%d",&a[i]);
    }
    for(int i=0;i<n;i++){
        for(int j=i+1;j<n;j++){
            if(a[i]==a[j]){
                printf("%d",a[i]);
                return 0;
            }
        }
    }
    return 0;
}
```

2-Finding Duplicates-O(n) Time Complexity,O(1) Space Complexity

```
#include <stdio.h>

int main() {
    int n, a[100], i;
    int slow, fast;

    scanf("%d", &n);

    for(i = 0; i < n; i++)
        scanf("%d", &a[i]);

    slow = a[0];
    fast = a[0];

    do {
        slow = a[slow];
        fast = a[a[fast]];
    } while(slow != fast);

    slow = a[0];

    while(slow != fast) {
        slow = a[slow];
        fast = a[fast];
    }

    printf("%d", slow);

    return 0;
}
```

*3-Print Intersection of 2 sorted arrays-O(m*n)Time Complexity,O(1) Space Complexity*

```
#include <stdio.h>

int main() {
    int T;
    scanf("%d", &T);

    while (T--) {
        int n1, n2;
        scanf("%d", &n1);

        int a[n1];
        for (int i = 0; i < n1; i++)
            scanf("%d", &a[i]);

        scanf("%d", &n2);

        int b[n2];
```

```
        for (int i = 0; i < n2; i++)
            scanf("%d", &b[i]);

        int i = 0, j = 0;

        while (i < n1 && j < n2) {
            if (a[i] == b[j]) {
                printf("%d ", a[i]);
                i++;
                j++;
            }
            else if (a[i] < b[j])
                i++;
            else
                j++;
        }

        printf("\n");
    }

    return 0;
}
```

4-Print Intersection of 2 sorted arrays-O(m+n)Time Complexity,O(1) Space Complexity

```
#include <stdio.h>

int main() {
    int T;
    scanf("%d", &T);

    while (T--) {
        int n1, n2;
        scanf("%d", &n1);

        int a[n1];
        for (int i = 0; i < n1; i++)
            scanf("%d", &a[i]);

        scanf("%d", &n2);

        int b[n2];
        for (int i = 0; i < n2; i++)
            scanf("%d", &b[i]);

        int i = 0, j = 0;

        while (i < n1 && j < n2) {
            if (a[i] == b[j]) {
                printf("%d ", a[i]);
                i++;
                j++;
            }
            else if (a[i] < b[j])
                i++;
            else
                j++;
        }

        printf("\n");
    }

    return 0;
}
```

5-Pair with Difference-O(n^2) Time Complexity,O(1) Space Complexity

```
#include <stdio.h>

int main() {
    int n, a[1000], k, i, j;

    scanf("%d", &n);

    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);
```

```
    scanf("%d", &k);

    for (i = 0; i < n; i++) {
        for (j = 0; j < n; j++) {
            if (i != j && a[j] - a[i] == k) {
                printf("1");
                return 0;
            }
        }
    }

    printf("0");

    return 0;
}
```

6-Pair with Difference-O(n) Time Complexity,O(1) Space Complexity

```
#include <stdio.h>

int main() {
    int n, a[1000], k;
    int i = 0, j = 1;

    scanf("%d", &n);

    for (int x = 0; x < n; x++)
        scanf("%d", &a[x]);

    scanf("%d", &k);

    while (j < n) {
        if (a[j] - a[i] == k && i != j) {
            printf("1");
            return 0;
        }
        else if (a[j] - a[i] < k) {
            j++;
        }
        else {
            i++;
        }

        if (i == j)
            j++;
    }

    printf("0");

    return 0;
}
```